



Nuevos tipos y características

Programación con JavaScript II



Actividad de Bienestar





Actividad Guiada

Parte 1

Duración: 75 minutos.



Temas

- | | |
|---|---|
| 1 | Manejadores de paquetes y empaquetados de módulos |
| 2 | Nuevos tipos y características |
| 3 | Asincronía |
| 4 | Ajax |
| 5 | API |
| 6 | Scope y This |
| 7 | Transpiladores |
| 8 | Pruebas |



Prework

- Leer detenidamente y comprender el material explicado en el tema 2.
- Practicar todos los ejemplos que se describen previamente en el tema 2.
- Revisar cada uno de los recursos adicionales que se proponen en el tema 2.



Para saber más

Lecturas

- Haverbeke, M. (2018). La vida secreta de los objetos. Recuperado de https://eloquentjs-es.thedojo.mx/06_object.html
- Lenguaje JS. (s.f.). Tipos de datos. Recuperado de <https://lenguajejs.com/javascript/fundamentos/tipos-de-datos/>



Para saber más

Videos

- Aprender. (2021, 25 de julio). Aprender con JavaScript - 98 Computed properties en JS 2021 [Archivo de video]. Recuperado de <https://www.youtube.com/watch?v=CaO8Mp1mtzQ>
- La Cocina del Código. (2019, 19 de septiembre). 2. TIPOS DE DATOS PRIMITIVOS en JAVASCRIPT | JS en ESPAÑOL [Archivo de video]. Recuperado de <https://www.youtube.com/watch?v=cC65D2q5f8I>
- Midulive. (2022, 18 de marzo). Funciones Generadoras en JavaScript. [Archivo de video]. Recuperado de <https://www.youtube.com/watch?v=wl6X2XFnNtY>



Ejercicio 1: Función en JavaScript



Objetivo del ejercicio

Crear una función en JavaScript que elimine los elementos duplicados de un arreglo, validando correctamente la entrada del usuario.



Ejercicio 1

Crea una función que, dado un arreglo de elementos, elimine los duplicados.

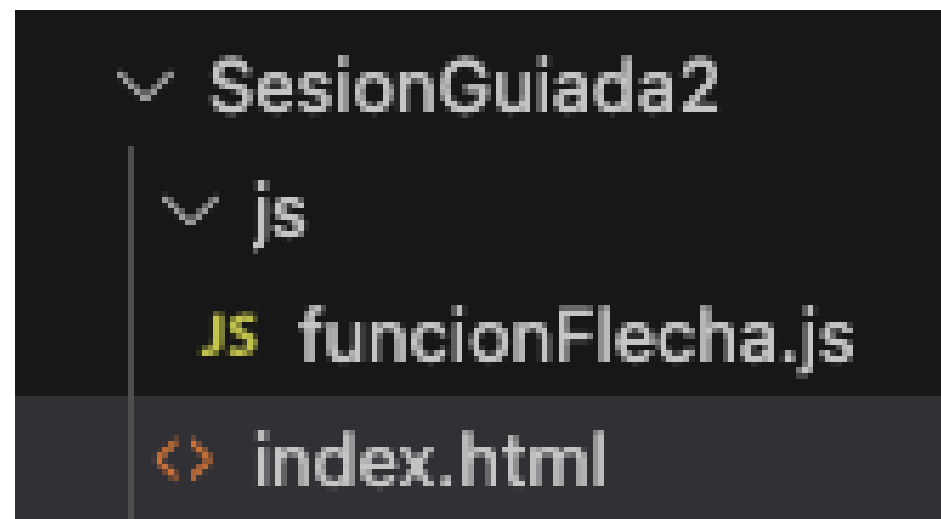
Por ejemplo: **MiFuncion**(["10","X","X","2","10",10,true, true]) devolverá ["10","X",2,10,true].

Valida que el arreglo se haya ingresado, que el valor ingresado sea un arreglo, que el arreglo no esté vacío y que el arreglo tenga al menos dos elementos.



Parte 1: Preparación del entorno

1. Dentro de la carpeta js, crea un nuevo archivo llamado:
funcionflecha.js
2. Enlaza el script con el archivo
index.html





¿Qué es una función flecha?

Una **función flecha** (arrow function) es una forma más corta y moderna de escribir funciones en JavaScript. Fue introducida en ES6 (ECMAScript 2015) para simplificar la sintaxis de las funciones y hacer el código más limpio y legible.



Estructuras básicas función flecha

```
// Función flecha básica sin parámetros
const nombreFuncion = () => {
  // Código a ejecutar
};

// Función flecha con un parámetro
const nombreFuncion = (parametro) => {
  // Código a ejecutar con 'parametro'
};

// Función flecha con varios parámetros
const nombreFuncion = (param1, param2) => {
  // Código a ejecutar con 'param1' y 'param2'
};
```



Parte 2: Creación función flecha

1. Crea la función flecha quitarDuplicados y pásale como parámetro default la variable arr, asignándole el valor indefinido.



Parte 2: Creación función flecha

1. Crea la función flecha quitarDuplicados y pásale como parámetro default la variable arr, asignándole el valor indefinido.

```
//  Función flecha quitarDuplicados con validaciones  
const quitarDuplicados = (arr = undefined) => { ...  
};
```



Parte 3: Validaciones

- Validar que se haya recibido un parámetro.
- Validar que el parámetro sea un arreglo (instanceof Array).
- Validar que el arreglo no esté vacío (`length > 0`).
- Validar que tenga al menos dos elementos (`length >= 2`).



Parte 3: Validaciones

```
// ✓ Función flecha quitarDuplicados con validaciones
const quitarDuplicados = (arr = undefined) => {
  // 1. Validar que el parámetro haya sido ingresado
  if (arr === undefined) return console.warn("No ingresaste un arreglo");

  // 2. Validar que sea un arreglo
  if (!(arr instanceof Array)) return console.error("El valor que ingresaste no es un arreglo");

  // 3. Validar que no esté vacío
  if (arr.length === 0) return console.error("El arreglo está vacío");

  // 4. Validar que tenga al menos dos elementos
  if (arr.length === 1) return console.error("El arreglo debe tener al menos dos elementos");
}
```



Parte 4: Eliminar duplicados

- Muestra en consola un **objeto** con dos propiedades:
 1. `original`: El arreglo original
 2. `sinDuplicados`: El arreglo sin duplicados usando `filter()`.
Pásale como parámetro una función flecha que reciba como parámetros el valor, el índice y `self`. La función flecha lo que hará es aplicar el método `indexOf(valor=== índice)`



Parte 4: Eliminar duplicados

```
// 5. Mostrar resultado con método filter
console.info({
  original: arr,
  sinDuplicados: arr.filter((valor, indice, self) => self.indexOf(valor) === indice)
});
```



Parte 5: Casos de Pruebas

```
quitarDuplicados(); // a. undefined  
quitarDuplicados({}); // b. objeto  
quitarDuplicados([]); // c. arreglo vacío  
quitarDuplicados([2]); // d. solo un elemento  
quitarDuplicados(["10", "X", "X", "2", "10", 10, true, true]); // e. duplicados variados
```



Parte 6: Mejora de filtrado con SET

- Para hacer la mejora, **comenta el bloque filter**
- Crea el bloque con Set.
- Muestra en consola un **objeto** con dos propiedades:
 1. `original`: El arreglo original
 2. `sinDuplicados`: El arreglo sin duplicados usando el objeto **Set**.



Parte 6: Mejora de filtrado con SET

```
// Alternativa mejorada con Set (comentada inicialmente)
console.info({
  original: arr,
  sinDuplicados: [...new Set(arr)]
});
```

Ejecuta nuevamente los casos de prueba



Ejercicio 2: Limpieza de Anagramas



Objetivo del ejercicio:

Crear una función que elimine anagramas de un arreglo, devolviendo solo una palabra representativa de cada grupo.



¿Qué es un anagrama?

- Palabras que usan **las mismas letras**, pero en **diferente orden**.

Ejemplos:

- “roma” – “amor” – “mora”
- “sopa” – “paso” – “sapo”
- “ropa” – “paro” – “proa”



Vamos a trabajar con:



toLowerCase() → convierte texto a minúsculas.



split("") → divide una palabra en letras.



sort() → ordena letras alfabéticamente.



join(' ') → une las letras en una sola palabra.



Object y Map → estructuras para guardar pares clave–valor.



Object.values(obj) → extrae los valores de un objeto.

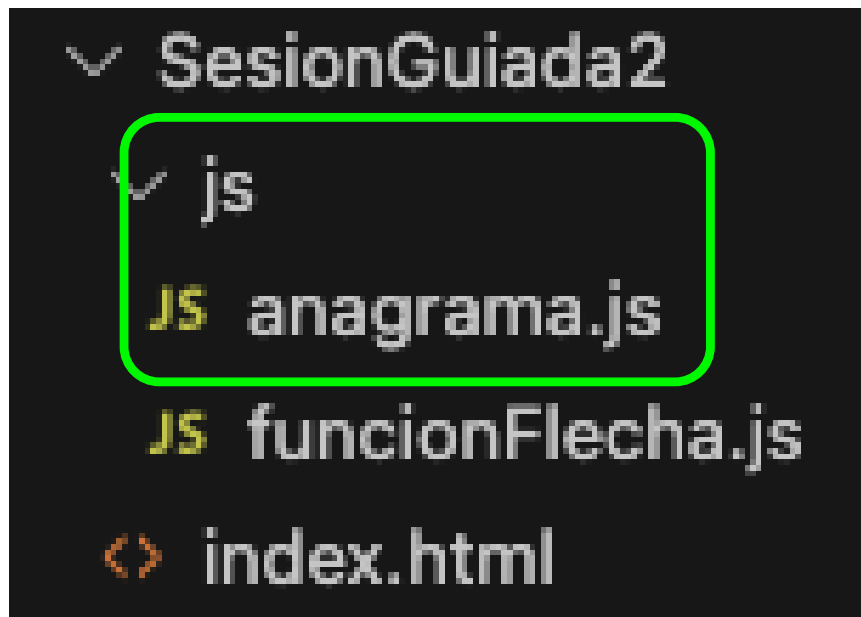


Array.from(mapa.values()) → convierte valores de un Map en arreglo.



Parte 1: Preparación del entorno

1. Dentro de la carpeta js, crea un nuevo archivo llamado:
anagrama.js
2. Enlaza el script con el archivo **index.html** previamente creado.





Parte 2: Visión con objeto plano

1. Dentro del archivo anagrama, crea la función **limpiaAnagrama**, la cual recibirá como parámetro el arreglo.
2. Declara un objeto vacío.



Parte 2: Visión con objeto plano

1. Dentro del archivo anagrama, crea la función `limpiaAnagrama`, la cual recibirá como parámetro el arreglo.
2. Declara un objeto vacío.

```
const limpiaAnagrama = (arr) => {  
  const obj = {};
```



Parte 2: Visión con objeto plano

3. Declara un **for** que va a iterar sobre el arreglo pasado como parámetro:
 - Declara la variable ordenado e iguálala al elemento del arreglo al que le vas a aplicar los métodos **toLowerCase()**, **split()**, **sort()**, **join()** para dividir la palabra por letras, ordenarlas y volverlas a unir.
 - Asigna al elemento del objeto iterado el valor de arreglo iterado de la siguiente manera: **obj[sorted]=arr[i]**.



Parte 2: Visión con objeto plano

```
const limpiaAnagrama = (arr) => {  
  const obj = {};  
  for (let i = 0; i < arr.length; i++) {  
    let ordenado = arr[i].toLowerCase().split('').sort().join('');  
    obj[ordenado] = arr[i];  
  }  
}
```



Parte 2: Visión con objeto plano

4. Con un return, utiliza el método values de la clase Object a la cual le pasarás como parámetro el objeto obj.
5. Declara el arreglo que se quiere evaluar.
6. En un alert, manda a llamar la función limpiaAnagrama(arreglo).



Parte 2: Visión con objeto plano

4. Con un return, utiliza el método values de la clase Object a la cual le pasarás como parámetro el objeto obj.

```
return Object.values(obj);
```

5. Declara el arreglo que se quiere evaluar.

```
const palabras = ["roma", "amor", "mora", "sopa", "paso", "sapo"];
```

6. En un alert, manda a llamar la función limpiaAnagrama(arreglo).

```
alert(limpiaAnagrama(palabras));
```



Parte 2: Visión con objeto plano

```
const limpiaAnagrama = (arr) => {  
  const obj = {};  
  for (let i = 0; i < arr.length; i++) {  
    let ordenado = arr[i].toLowerCase().split('').sort().join('');  
    obj[ordenado] = arr[i];  
  }  
  return Object.values(obj);  
};  
  
const palabras = ["roma", "amor", "mora", "sopa", "paso", "sapo"];  
alert(limpiaAnagrama(palabras));
```



Parte 3: Visión con tipo de dato Map

1. Declara la función flecha **limpiaAnagramaMap**, la cual recibe como parámetro un arreglo.
2. Declara mapa, que va a ser igual a un nuevo **Map()**. Creamos un Map vacío para claves únicas.



Parte 3: Visión con tipo de dato Map

1. Declara la función flecha **limpiaAnagramaMap**, la cual recibe como parámetro un arreglo.
2. Declara la variable **mapa**, que va a ser igual a un nuevo **Map()**.

```
JS anagrama.js > ...  
  const limpiaAnagramaMap = (arr) => {  
    const mapa = new Map();
```



Parte 3: Visión con tipo de dato Map

3. Crea un **for** of:

- a. Evalúa cada palabra (variable declarada en el for) del arreglo pasado como parámetro.
- b. Declara la variable ordenado e iguálala al elemento del arreglo a la que le vas a aplicar los métodos **toLowerCase()**, **split()**, **sort()**, **join()** para dividir la palabra por letras, ordenarlas y volverlas a unir.
- c. Utiliza la función set de mapa para fijar ordenado y palabra como clave, valor.



Parte 3: Visión con tipo de dato Map

```
JS anagrama.js > ...  
const limpiaAnagramaMap = (arr) => {  
  const mapa = new Map();  
  for (const palabra of arr) {  
    let ordenado = palabra.toLowerCase().split('').sort().join('');  
    mapa.set(ordenado, palabra);  
  }  
}
```



Parte 3: Visión con tipo de dato Map

4. Con un **return**, utiliza el método `from` de la clase `Array` para pasar como parámetro los valores de mapa.
5. Declara el arreglo que se quiere evaluar.
6. En un `alert`, manda a llamar la función **`limpiaAnagramaMapa(arreglo)`**.



Parte 3: Visión con tipo de dato Map

4. Con un **return**, utiliza el método `from` de la clase `Array` para pasar como parámetro los valores de mapa.

```
return Array.from(mapa.values());
```

5. Declara el arreglo que se quiere evaluar.

```
const palabrasMap = ["ropa", "paro", "proa", "amor", "roma",  
"mora", "sopa", "paso", "sapo"];
```

6. En un `alert`, manda a llamar la función

```
alert(limpiaAnagramaMap(palabrasMap));
```



Parte 3: Visión con tipo de dato Map

```
// Función que elimina anagramas utilizando un Map (estructura más moderna)
const limpiaAnagramaMap = (arr) => {
  const mapa = new Map(); // Creamos un Map vacío para claves únicas

  for (const palabra of arr) {
    // Igual que antes, convertimos a minúsculas, ordenamos y unimos para normalizar
    let ordenado = palabra.toLowerCase().split('').sort().join('');
    // Insertamos en el Map, usando como clave la palabra ordenada
    mapa.set(ordenado, palabra); // Si la clave ya existe, se sobrescribe
  }

  // Convertimos el Map a arreglo y retornamos los valores (una palabra por grupo)
  return Array.from(mapa.values());
};

// Arreglo de palabras con más grupos de anagramas
const palabrasMap = ["ropa", "paro", "proa", "amor", "roma",
  "mora", "sopa", "paso", "sapo"];
// Mostramos el resultado en pantalla
alert(limpiaAnagramaMap(palabrasMap));
```



Tabla comparativa

Aspecto	Objeto Plano ({})	Map()
Sintaxis tradicional	Sí	Más moderna
Claves ordenadas	No necesariamente	Sí (mantiene orden de inserción)
Métodos útiles	Object.values()	mapa.set(), mapa.values()
Rendimiento	Rápido	Similar, más flexible



Receso

Duración: 10 minutos.



Actividad Guiada

Parte 2

Duración: 75 minutos.



Ejercicio 3: Generadores e Iteradores



Objetivo del ejercicio

Modificar la función que genera el cuadrado de un número usando: `setTimeout`, funciones generadores e iteradores



¿Qué es una función generadora?

```
function* nombreGenerador() {  
  yield valor1;  
  yield valor2;  
}
```

Características:

- Se define con function*
- Usa yield para **entregar un valor** y **pausar** ejecución
- Devuelve un **iterador** que se controla con .next() o for...of



¿Qué es un iterador?

- Un objeto que permite recorrer una colección paso a paso
- Cada llamada a **.next()** devuelve { **value**, **done** }
- Con **for...of**, obtenemos automáticamente los valores generados por **yield**



Parte 1: Preparación del entorno

1. Dentro de la carpeta js, crea un nuevo archivo llamado:
generadoresIteradores.js
2. Enlaza el script con el archivo
index.html



Parte 2. Función cuadrado con setTimeout

1. En el archivo, crea la función **cuadrado** y pasa como parámetro un valor. `function cuadrado(valor){ }`
2. Dentro del cuerpo de la consola, ejecuta un **setTimeout** que recibirá como parámetro una función anónima, lo único que hará es regresar la impresión en consola (un objeto con el valor y el resultado) y como temporizador utiliza un número aleatorio multiplicado por 1000. `Math.random() * 1000`
3. Regresa el objeto formado por **valor** y **resultado**.



Parte 2. Función cuadrado con setTimeout

```
function cuadrado(valor) {  
  setTimeout(() => {  
    console.info({ valor, resultado: valor * valor });  
  }, Math.random() * 1000);  
}
```



Parte 3. Función generadora

Text

1. Crea una función generadora llamada **generador** sin parámetros.
2. Dentro del cuerpo de la función, imprime el texto “Inicia Generador”.
3. Luego utiliza un yield pasándole como parámetro 0, otro pasándole 1 y así sucesivamente hasta 5.
4. Imprime en consola el texto “Termina Generador”.



Parte 3. Función generadora

```
function* generador() {  
  console.log("Inicia Generador");  
  yield 0;  
  yield 1;  
  yield 2;  
  yield 3;  
  yield 4;  
  yield 5;  
  console.log("Termina Generador");  
}
```



Parte 4. Iteraciones

1. Declara la función **gen** en la que asignarás la llamada a la función generador.
2. Crea un **for/of** en el que recorras la variable **gen** para cada **yield** e imprimas en consola cada elemento **yield**.



Parte 4. Iteraciones

1. Declara la constante **gen** en la que asignarás la llamada a la función generador.
2. Crea un **for/of** en el que recorras la variable **gen** para cada **yield** e imprimas en consola cada elemento **yield**.

```
const gen = generador();  
for (const valor of gen) {  
  | cuadrado(valor);  
}
```



Ejercicio 4: Uso de Proxy en JavaScript



Objetivo del ejercicio

Comprender el uso de Proxy en JavaScript para interceptar y validar el acceso y modificación de propiedades en objetos.



¿Qué es un Proxy?

Un **Proxy** en JavaScript es un objeto que permite interceptar y redefinir operaciones fundamentales como lectura y escritura de propiedades.

```
const proxy = new Proxy(objetivo, manejador);
```

- **objetivo**: el objeto original.
- **manejador**: define qué hacer al leer o escribir propiedades (get, set, etc.).



Parte 1: Preparación del entorno

1. Dentro de la carpeta js, crea un nuevo archivo llamado:
proxy.js
2. Enlaza el script con el archivo
index.html



Parte 2: Crear el objeto original

Crea el objeto base llamado persona con las propiedades nombre, apellido y edad. Establece los valores nombre y persona como vacíos y el valor de edad como 0.



Parte 2: Crear el objeto original

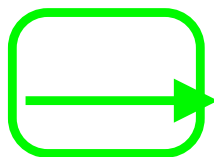
Crea el objeto base llamado persona con las propiedades nombre, apellido y edad. Establece los valores nombre y persona como vacíos y el valor de edad como 0.

```
// Objeto original
const persona = {
  nombre: '',
  apellido: '',
  edad: 0
};
```



Parte 3: Crear el manejador (handler)

1. Crea el manejador, el cual es un objeto que recibe un parámetro set, que, a su vez, recibe dentro tres parámetros: el objeto, las propiedades que se desean evaluar (nombre, apellido y edad) y el valor.
2. Al objeto en cuestión, en cada una de las tres propiedades modificadas, asígnale el valor que el usuario está proporcionando.





Parte 3: Crear el manejador (handler)

```
const manejador = {  
  set(obj, prop, valor) {  
    // Validar que la propiedad sea permitida  
    const propiedadesPermitidas = ['nombre', 'apellido', 'edad'];  
  
    if (propiedadesPermitidas.includes(prop)) {  
      // Asignar el valor al objeto  
      obj[prop] = valor;  
    } else {  
      // Mostrar advertencia si la propiedad no es válida  
      console.error(`❌ La propiedad "${prop}" no está permitida en el objeto persona.`);  
    }  
  
    // Importante: retornar true para que el proxy no lance un error  
    return true;  
  }  
};
```



Parte 4: Crear el Proxy y asignar valores válidos

1. Crea un nuevo proxy y asígnale un nombre. Dicho proxy recibirá como parámetros los objetos persona y manejador.
2. Asigna valores (nombre= “Juan”, apellido= “Pérez”, edad=35) a la propiedad nombre, apellido y edad.
3. Envía a la consola el proxy creado.



Parte 4: Crear el Proxy y asignar valores válidos

1. Crea un nuevo proxy y asígnale un nombre. Dicho proxy recibirá como parámetros los objetos persona y manejador.
2. Asigna valores (nombre= "Juan", apellido= "Pérez", edad=35) a la propiedad nombre, apellido y edad.
3. Envía a la consola el proxy creado.

```
// Crear el proxy
const juan = new Proxy(persona, manejador);

// Asignar valores válidos
juan.nombre = "Juan";
juan.apellido = "Pérez";
juan.edad = 35;

console.info("Proxy juan:", juan);
```



Parte 5: Agrega propiedad no existente

Ahora crea una propiedad `juan.twitter=@juanperez`.
Observa el resultado en la consola.

```
// Intentar asignar una propiedad no válida  
juan.twitter = "@juanperez"; // No se asignará
```



Parte 5: Agrega propiedad no existente

Ahora crea una propiedad `juan.twitter=@juanperez`.
Observa el resultado en la consola.

```
// Intentar asignar una propiedad no válida  
juan.twitter = "@juanperez"; // No se asignará
```

► ✖ La propiedad "twitter" no está permitida en el objeto persona.



Parte 6: Impresión en consola objeto persona

```
console.info("Objeto persona:", persona);
```

```
Objeto persona: {nombre: 'Juan', apellido: 'Pérez', edad: 35} ⓘ  
  apellido: "Pérez"  
  edad: 35  
  nombre: "Juan"
```

¿Qué ves? ¿Por qué tiene esos valores?



Parte 7: Validación propiedades permitidas

1. Dentro del set del manejador, evalúa que exista en la lista del objeto original una propiedad, esto lo puedes realizar con el método **indexOf**.
2. En caso de que regrese un -1 , retorna por medio de un error en la consola “La propiedad `${propiedad_en_cuestion}` no existe en el objeto persona”.



Parte 7: Validación propiedades permitidas

```
const manejador = {
  set(obj, prop, valor) {
    // Lista de propiedades permitidas
    const propiedadesPermitidas = ["nombre", "apellido", "edad"];

    // Validar si la propiedad está permitida
    if (propiedadesPermitidas.indexOf(prop) === -1) {
      console.error(`La propiedad "${prop}" no existe en el objeto persona`);
      return false; // Importante: regresar false para que el Proxy sepa que falló
    }

    // Si es válida, asigna el valor
    obj[prop] = valor;
    return true; // Se pudo asignar correctamente
  }
};
```



Parte 8: Observa el resultado

1. Guarda cambios
2. Carga de nuevo index.html
3. Observa el resultado en la consola.



Parte 8: Observa el resultado

1. Guarda cambios
2. Carga de nuevo index.html
3. Observa el resultado en la consola.

► La propiedad "twitter" no existe en el objeto persona



Parte 9: Validación con expresión regular

1. Crea una nueva validación que valide el nombre y apellido y que solo acepte letras y espacios.
2. Opcional, utilizar la siguiente expresión regular:
`(/^[A-Za-zÑñÁáÉéÍíÓóÚúÜü\s]+$ /g.test(valor))`



Parte 9: Validación con expresión regular

1. Crea una nueva validación que valide el nombre y apellido y que solo acepte letras y espacios.
2. Opcional, utilizar la siguiente expresión regular:
(/^[A-Za-zÑñÁáÉéÍíÓóÚúÜü\s]+\$/g.test(valor))

```
// Validación de letras y espacios para nombre y apellido
if ((prop === "nombre" || prop === "apellido") &&
    !/^[A-Za-zÑñÁáÉéÍíÓóÚúÜü\s]+$/g.test(valor)) {
  console.error(`❌ El valor de "${prop}" no es válido. Solo se permiten letras y espacios.`);
  return;
}
```



Validaciones clave set:

- ✗ Rechaza propiedades no listadas: `indexOf(prop) === -1`
- ✗ Rechaza nombres/apellidos inválidos con RegExp
- ✓ Acepta y asigna solo propiedades válidas con valores válidos



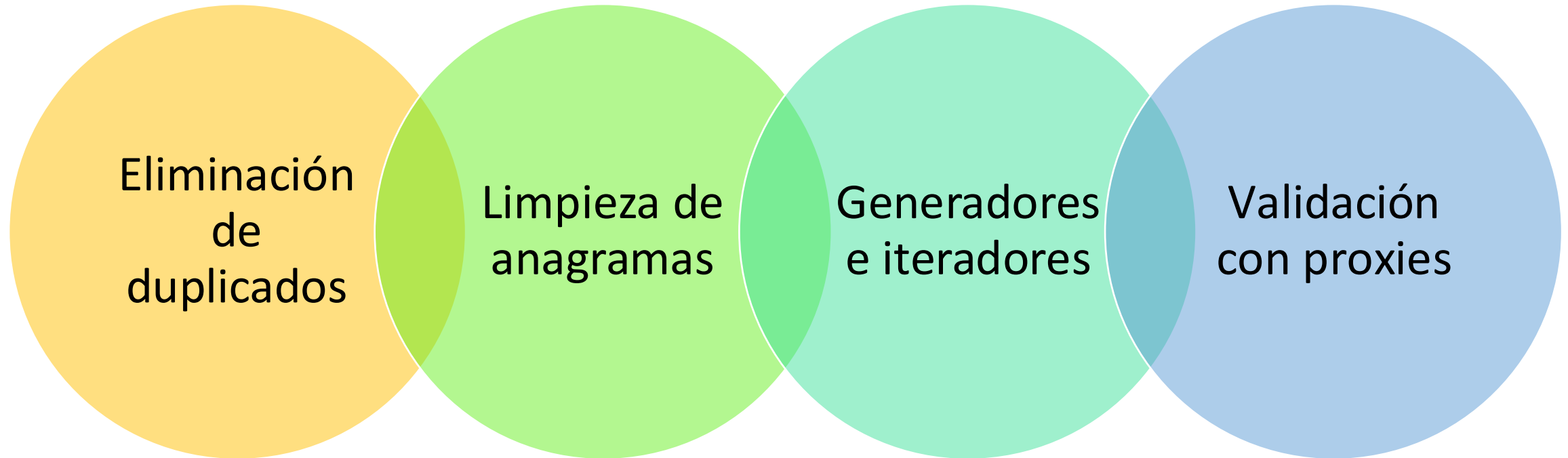
Cierre

Duración: 10 minutos.



Resumen general

Hoy aplicamos conceptos clave de JavaScript a través de 4 ejercicios prácticos:





Ejercicio 1 – Eliminar duplicados



Objetivo: Retornar un objeto que contenga el arreglo original y una versión sin duplicados.



Conceptos clave:

- Funciones flecha () => {}
- Validación con instanceof Array
- Métodos: .filter(), .indexOf(), Set, operador ...
- Buenas prácticas: desarrollo modular, casos de prueba



Ejercicio 2 – Anagramas con Map y objeto plano

 **Objetivo:** Agrupar palabras anagramas y retornar una sola por grupo.

Técnicas empleadas:

-  for y for...of loops
-  Normalización de palabras (`toLowerCase().split().sort().join()`)
-  Uso de claves únicas en:
 - Objetos planos: `obj[ordenado] = palabra`
 - Map: `map.set(ordenado, palabra)`



Ejercicio 3 – Generadores e iteradores



Objetivo: Usar funciones generadoras para emitir valores secuenciales y simular asincronía.



Conceptos:

- `function* generador() { yield valor; }`
- Iteración con `for...of`
- `setTimeout()` con tiempos aleatorios
- Return de objetos: `{ valor, resultado }`



Ejercicio 4 – Proxy y validación de propiedades



Objetivo: Controlar el acceso a propiedades de un objeto con Proxy.



Técnicas:

- Proxy(objeto, manejador)
- set(obj, prop, valor) con validación
- Validación con .indexOf() y expresiones regulares
- Mensajes de error personalizados



¿Qué aprendimos?

- ✓ Definir funciones modernas
- ✓ Validar datos y estructuras
- ✓ Manipular arreglos y strings
- ✓ Usar Set, Map, Proxy, Generator

 Aplicamos programación funcional y orientada a objetos

 Probamos distintos enfoques para solucionar un mismo problema